

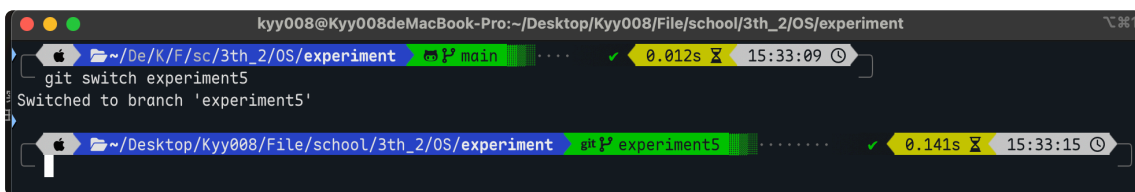
# 2026 春《操作系统》课程实验报告

## 实验 5

23301037 客昱阳

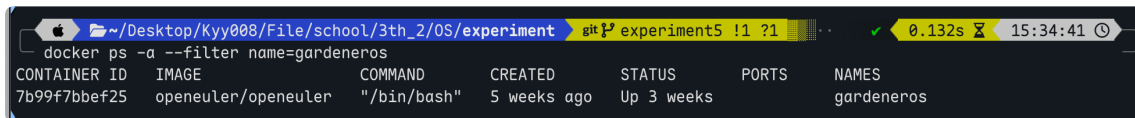
### 一.准备工作

- 将分支切换到实验四所需的 `experiment5` 分支



```
kyy008@Kyy008deMacBook-Pro: ~/Desktop/Kyy008/File/school/3th_2/OS/experiment
git switch experiment5
Switched to branch 'experiment5'
```

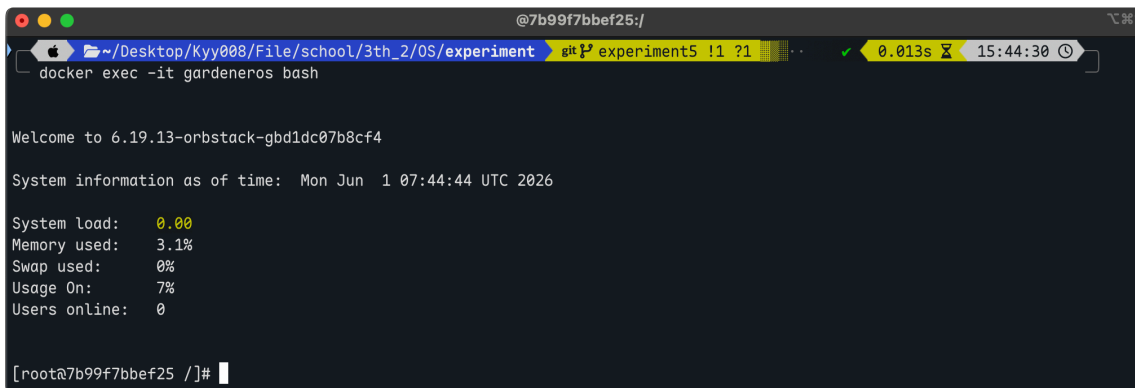
- 检查一下上次环境配置的 docker 容器已经启动



```
docker ps -a --filter name=gardeneros
```

| CONTAINER ID | IMAGE               | COMMAND     | CREATED     | STATUS     | PORTS | NAMES      |
|--------------|---------------------|-------------|-------------|------------|-------|------------|
| 7b99f7bbef25 | openeuler/openeuler | "/bin/bash" | 5 weeks ago | Up 3 weeks |       | gardeneros |

- 进入容器



```
docker exec -it gardeneros bash
```

```
Welcome to 6.19.13-orbstack-gbd1dc07b8cf4

System information as of time: Mon Jun 1 07:44:44 UTC 2026

System load: 0.00
Memory used: 3.1%
Swap used: 0%
Usage On: 7%
Users online: 0

[root@7b99f7bbef25 /]#
```

### 二.实验步骤

#### 1.时钟中断与计时器

- 首先创建 `timer` 子模块, 用来读取 RISC-V 的 `time` 寄存器。这个函数可以获得当前计时器的值, 后续设置时钟中断和实现 `get_time` 系统调用都会用到它。

Bash

```
1 cd /mnt
2 vim os/src/timer.rs
```

写入以下内容：



```
"os/src/timer.rs" [New] 0,0-1 ALL
use riscv::register::time;

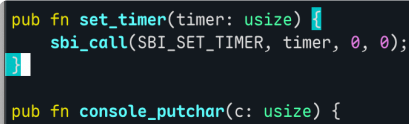
pub fn get_time() -> usize {
    time::read()
}
```

- 修改 `sbi.rs`，增加设置下一次时钟中断的 SBI 调用。`set_timer` 会设置 `mtimecmp`，当 `mtime` 达到该值时触发一次时钟中断。

Bash

```
1 vim os/src/sbi.rs
```

在 `console_putchar` 前加入：



```
pub fn set_timer(timer: usize) {
    sbi_call(SBI_SET_TIMER, timer, 0, 0);
}

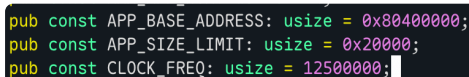
pub fn console_putchar(c: usize) {
```

- 在 `config.rs` 中增加时钟频率常量，后续设置时钟中断和计算毫秒时间都会用到。

Bash

```
1 vim os/src/config.rs
```

在文件末尾加入：



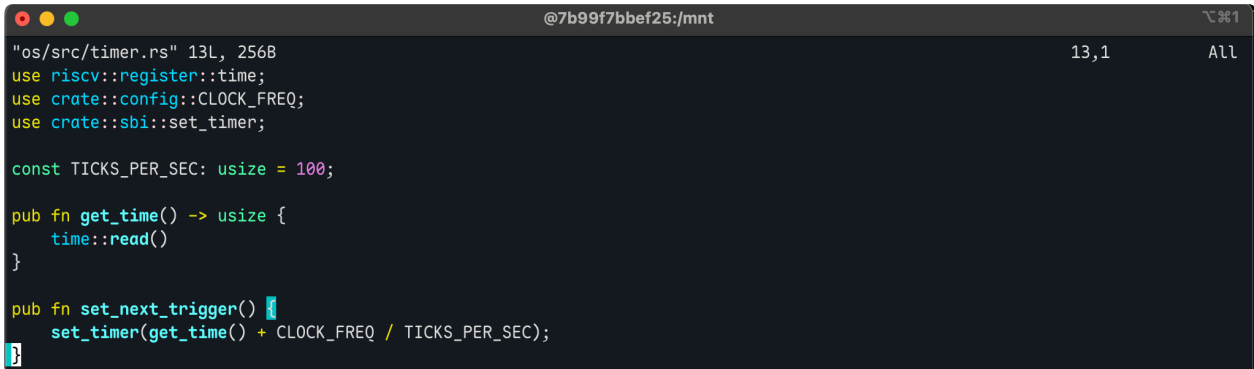
```
pub const APP_BASE_ADDRESS: usize = 0x80400000;
pub const APP_SIZE_LIMIT: usize = 0x20000;
pub const CLOCK_FREQ: usize = 12500000;
```

- 继续完善 `timer.rs`，封装设置下一次时钟中断的函数。每次调用 `set_next_trigger`，都会通过 SBI 设置下一次触发时钟中断的时间。

Bash

```
1 vim os/src/timer.rs
```

修改为以下内容：



```
@7b99f7bbef25:/mnt
"os/src/timer.rs" 13L, 256B 13,1 All
use riscv::register::time;
use crate::config::CLOCK_FREQ;
use crate::sbi::set_timer;

const TICKS_PER_SEC: usize = 100;

pub fn get_time() -> usize {
    time::read()
}

pub fn set_next_trigger() {
    set_timer(get_time() + CLOCK_FREQ / TICKS_PER_SEC);
}
```

- 继续在 `timer.rs` 中封装毫秒级时间函数。`get_time_ms` 会把当前计时器值转换成毫秒，后续 `get_time` 系统调用会使用它。

Bash

```
1 vim os/src/timer.rs
```

在文件末尾加入：



```
const MSEC_PER_SEC: usize = 1000;

pub fn get_time_ms() -> usize {
    time::read() / (CLOCK_FREQ / MSEC_PER_SEC)
}
```

- 在内核中实现 `get_time` 系统调用，返回当前毫秒时间。

Bash

```
1 vim os/src/syscall/process.rs
```

将文件修改为：

```
@7b99f7bbef25:/mnt
use crate::timer::get_time_ms;
use crate::task::{
    exit_current_and_run_next,
    suspend_current_and_run_next,
};
pub fn sys_exit(exit_code: i32) -> ! {
    println!("[kernel] Application exited with code {}", exit_code);
    exit_current_and_run_next();
    panic!("Unreachable in sys_exit!");
}
pub fn sys_yield() -> isize {
    suspend_current_and_run_next();
    0
}
pub fn sys_get_time() -> isize {
    get_time_ms() as isize
}
```

- 修改系统调用分发，加入 `get_time` 系统调用编号和处理分支。

Bash

```
1 vim os/src/syscall/mod.rs
```

将文件修改为：

```
@7b99f7bbef25:/mnt
const SYSCALL_WRITE: usize = 64;
const SYSCALL_EXIT: usize = 93;
const SYSCALL_YIELD: usize = 124;
const SYSCALL_GET_TIME: usize = 169;

mod fs;
mod process;

use fs::*;
use process::*;

pub fn syscall(syscall_id: usize, args: [usize; 3]) -> isize {
    match syscall_id {
        SYSCALL_WRITE => sys_write(args[0], args[1] as *const u8, args[2]),
        SYSCALL_EXIT => sys_exit(args[0] as i32),
        SYSCALL_YIELD => sys_yield(),
        SYSCALL_GET_TIME => sys_get_time(),
        _ => panic!("Unsupported syscall_id: {}", syscall_id),
    }
}
```

## 2.修改应用程序

### (1)增加 get\_time 系统调用

- 在用户态系统调用封装中增加 `get_time` 系统调用，用来从内核获取当前毫秒时间。

Bash

```
1 vim user/src/syscall.rs
```

添加如下代码：

```
pub fn sys_yield() -> isize {
    "user/src/syscall.rs" 32L, 723B
    use core::arch::asm;

    const SYSCALL_WRITE: usize = 64;
    const SYSCALL_EXIT: usize = 93;
    const SYSCALL_YIELD: usize = 124;
    const SYSCALL_GET_TIME: usize = 169;
```

```
    syscall(SYSCALL_YIELD, [0, 0, 0])
}

pub fn sys_get_time() -> isize {
    syscall(SYSCALL_GET_TIME, [0, 0, 0])
}

-- INSERT --
```

- 在用户库中封装 `get_time`，让应用程序可以直接调用 `get_time()` 获取当前毫秒时间。

Bash

```
1 vim user/src/lib.rs
```

在 `yield_` 函数后面加入：

```
pub fn yield_() -> isize {
    sys_yield()
}

pub fn get_time() -> isize {
    sys_get_time()
}
```

## (2)实现新的测试应用

- 删除实验 4 中的测试应用，创建实验 5 的第一个测试应用 `00power_3.rs`。该程序用于持续计算 3 的幂，后续用来观察抢占式调度效果。

Bash

```
1 rm user/src/bin/*.rs
2 vim user/src/bin/00power_3.rs
```

```
@7b99f7bbef25:/mnt
"user/src/bin/00power_3.rs" [New] 0,0-1 All
extern crate user_lib;

const LEN: usize = 100;

#[unsafe(no_mangle)]
fn main() -> i32 {
    let p = 3u64;
    let m = 998244353u64;
    let iter: usize = 200000;
    let mut s = [0u64; LEN];
    let mut cur = 0usize;
    s[cur] = 1;
    for i in 1..=iter {
        let next = if cur + 1 == LEN { 0 } else { cur + 1 };
        s[next] = s[cur] * p % m;
        cur = next;
        if i % 10000 == 0 {
            println!("power_3 [{} / {}]", i, iter);
        }
    }
    println!("{}", s[cur]);
    println!("Test power_3 OK!");
}
0
```

- 创建第二个测试应用 `01power_5.rs`，具体代码略，见 git 仓库。同理创建第三个测试应用 `02power_7.rs`

Bash

```
1 vim user/src/bin/01power_5.rs
```

```
[root@7b99f7bbef25 mnt]# vim user/src/bin/01power_5.rs
[root@7b99f7bbef25 mnt]#
```

Bash

```
1 vim user/src/bin/02power_7.rs
```

```
[root@7b99f7bbef25 mnt]# vim user/src/bin/02power_7.rs
[root@7b99f7bbef25 mnt]#
```

- 创建第四个测试应用 `03sleep.rs`。该程序通过 `get_time()` 获取当前时间，并在等待期间不断调用 `yield_()` 主动让出 CPU。

Bash

```
1 vim user/src/bin/03sleep.rs
```

写入以下代码：

```
@7b99f7bbef25:/mnt
"user/src/bin/03sleep.rs" 17L, 310B      17,1      All
#![no_std]
#![no_main]
#[macro_use]
extern crate user_lib;

use user_lib::{get_time, yield_};

#[unsafe(no_mangle)]
fn main() -> i32 {
    let current_timer = get_time();
    let wait_for = current_timer + 3000;
    while get_time() < wait_for {
        yield_();
    }
    println!("Test sleep OK!");
    0
}
```

### 3. 抢占式调度

- 修改 `trap/mod.rs`，增加时钟中断处理。发生时钟中断时，内核会设置下一次时钟中断，并切换到下一个任务，从而实现抢占式调度。

Bash

```
1 vim os/src/trap/mod.rs
```

具体代码略，见 git 仓库

```
[root@7b99f7bbef25 mnt]# vim os/src/trap/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `main.rs`，加入 `timer` 模块，并在运行第一个任务前开启时钟中断、设置第一次时钟触发时间。

Bash

```
1 vim os/src/main.rs
```

进行如下修改：

```
mod trap;
mod loader;
mod config;
mod task;
mod timer;
```

```
#[unsafe(no_mangle)]
pub fn rust_main() -> ! {
    clear_bss();
    println!("[kernel] Hello, world!");
    trap::init();
    trap::enable_timer_interrupt();
    timer::set_next_trigger();
    loader::load_apps();
    task::run_first_task();
    panic!("Unreachable in rust_main!");
}
```

## 4.编译并运行

重新编译用户程序。实验 5 的四个应用程序会分别被链接到不同地址，并生成对应的 `.bin` 文件。

Bash

```
1 cd /mnt/user
2 make build
```

```
@7b99f7bbef25:/mnt/user
[root@7b99f7bbef25 mnt]# cd /mnt/user
make build
  Compiling user_lib v0.1.0 (/mnt/user)
  Finished `release` profile [optimized] target(s) in 0.25s
[build.py] application 00power_3 start with address 0x8040000
  Compiling user_lib v0.1.0 (/mnt/user)
  Finished `release` profile [optimized] target(s) in 0.03s
[build.py] application 01power_5 start with address 0x8042000
  Compiling user_lib v0.1.0 (/mnt/user)
  Finished `release` profile [optimized] target(s) in 0.02s
[build.py] application 02power_7 start with address 0x8044000
  Compiling user_lib v0.1.0 (/mnt/user)
  Finished `release` profile [optimized] target(s) in 0.02s
[build.py] application 03sleep start with address 0x8046000
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/00power_3 --strip-all -O binary target/riscv64gc-unknown-none-elf/release/00power_3.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/01power_5 --strip-all -O binary target/riscv64gc-unknown-none-elf/release/01power_5.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/02power_7 --strip-all -O binary target/riscv64gc-unknown-none-elf/release/02power_7.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/03sleep --strip-all -O binary target/riscv64gc-unknown-n
```

Bash

```
1 cd /mnt/os
2 make run
```

```
power_3 [100000/200000]
power_3 [170000/200000]
power_3 [180000/200000]
power_3 [190000/200000]
power_3 [200000/200000]
3^200000 = 871008973
Test power_3 OK!
```

```
power_5 [150000/200000]
power_5 [160000/200000]
power_5 [170000/200000]
power_5 [180000/200000]
power_5 [190000/200000]
power_5 [200000/200000]
5^200000 = 670295496
Test power_5 OK!
```

```
power_7 [150000/200000]
power_7 [160000/200000]
power_7 [170000/200000]
power_7 [180000/200000]
power_7 [190000/200000]
power_7 [200000/200000]
7^200000 = 277895943
Test power_7 OK!
[kernel] Application exited with code 0
Test sleep OK!
[kernel] Application exited with code 0
Panicked at src/task/mod.rs:93 All applications completed!
```

至此，支持分时多任务和抢占式调度的操作系统实现完成。

## 三.问题回答

### 1.分析分时多任务是如何实现的

内核通过 timer 模块读取当前时间，并设置下一次时钟中断触发时间。

Rust

```
1 pub fn set_next_trigger() {  
2     set_timer(get_time() + CLOCK_FREQ / TICKS_PER_SEC);  
3 }
```

这段代码通过 SBI 设置下一次时钟中断。当时间到达设定值后，CPU 会进入内核的 Trap 处理流程。

在运行第一个任务前，内核需要开启时钟中断，并设置第一次触发时间。

Rust

```
1 trap::enable_timer_interrupt();  
2 timer::set_next_trigger();
```

这样用户程序运行时，内核可以周期性收到时钟中断，为后续任务切换提供条件。

当时钟中断发生时，内核会重新设置下一次时钟中断，并切换到下一个任务。

Rust

```
1 Trap::Interrupt(Interrupt::SupervisorTimer) => {  
2     set_next_trigger();  
3     suspend_current_and_run_next();  
4 }
```

因此，分时多任务是通过周期性时钟中断实现的。每个任务运行一小段时间后，内核通过中断重新获得 CPU，并调度下一个任务运行。

### 2.分析抢占式调度是如何设计和实现的

抢占式调度的核心是任务不需要主动调用 `yield_()`，内核可以通过时钟中断主动打断当前任务。

Rust

```
1 pub fn enable_timer_interrupt() {  
2     unsafe {  
3         sie::set_stimer();  
4     }  
5 }
```

该函数开启 `Supervisor Timer Interrupt`，使 CPU 能够响应时钟中断。

发生时钟中断后，Trap 处理函数会调用 `suspend_current_and_run_next()`，把当前任务挂起，并切换到下一个就绪任务。

Rust

```
1 Trap::Interrupt(Interrupt::SupervisorTimer) => {  
2     set_next_trigger();  
3     suspend_current_and_run_next();  
4 }
```

这里当前任务会从 Running 状态变回 Ready，然后任务管理器选择下一个 Ready 状态的任务继续执行。

任务切换仍然依赖实验 4 中实现的 `__switch`。它保存当前任务上下文，并恢复下一个任务上下文。

Rust

```
1 sd ra, 0(sp)  
2 ld sp, 0(a1)  
3 ld ra, 0(sp)  
4 ret
```

因此，抢占式调度是在实验 4 任务切换机制的基础上，增加时钟中断触发调度。任务即使不主动让出 CPU，也会被内核周期性切换出去。

### 3.对比上个实验实现的协作式调度与本实验实现的抢占式调度

实验 4 中实现的是协作式调度。应用程序需要主动调用 `yield_()`，通过系统调用让出 CPU。

Rust

```
1 pub fn sys_yield() -> isize {  
2     suspend_current_and_run_next();  
3     0  
4 }
```

如果应用程序一直不调用 `yield_()`，内核就无法主动切换到其他任务。

实验 5 中实现的是抢占式调度。内核通过时钟中断周期性获得 CPU 控制权，并主动切换任务。

Rust

```
1 Trap::Interrupt(Interrupt::SupervisorTimer) => {  
2     set_next_trigger();  
3     suspend_current_and_run_next();  
4 }
```

这种方式不依赖应用程序主动让出 CPU，因此更接近真实操作系统中的分时调度。

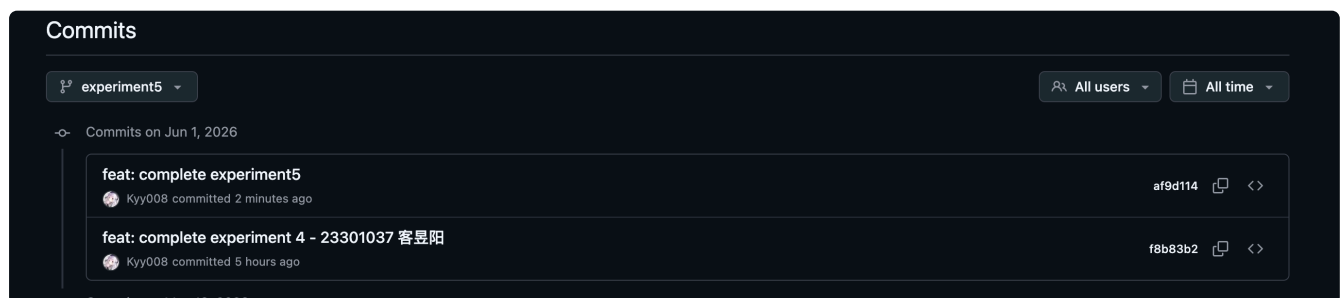
两种调度方式都使用同一套任务管理和上下文切换机制。

Rust

```
1 suspend_current_and_run_next();
```

而区别在于触发任务切换的方式不同：协作式调度由应用程序主动触发，抢占式调度由时钟中断触发。前者实现简单，但依赖应用程序配合；后者更加公平，可以避免某个任务长时间独占 CPU。

## 四.Git 提交截图



## 五.其他说明

无